

|                    |                                                           |                               |                                           |
|--------------------|-----------------------------------------------------------|-------------------------------|-------------------------------------------|
| <b>Asignatura:</b> | Programación Back End (TI3V41 / V-FB50)                   | <b>Estudiante:</b>            | Alexis Ferrada                            |
| <b>Docente:</b>    | Javier Eduardo Gutiérrez Osorio                           | <b>Puntaje Interrogación:</b> | <b>30 puntos</b> (Ponderación total: 35%) |
| <b>Proyecto:</b>   | <b>Proyecto_BD</b> (Django 5.2 + MySQL / SQLite + Docker) | <b>Repositorio GitHub:</b>    | github.com/alexispferrada-wq/proyecto_bd  |

## 1. Arquitectura General del Proyecto (Patrón MVT)

Django implementa el patrón de arquitectura **MVT (Model - View - Template)**, una variante del clásico MVC (Model-View-Controller):

- **Modelo ( `models.py` )**: Capa de datos. Define la estructura de las tablas mediante clases Python y el ORM. Maneja la comunicación con la base de datos sin escribir SQL manual.
- **Vista ( `views.py` )**: Capa de lógica de negocio. Recibe una solicitud HTTP ( `request` ), interactúa con los modelos para leer o guardar datos y devuelve una respuesta HTTP (renderizando un template o redirigiendo).
- **Plantilla ( `templates/*.html` )**: Capa de presentación visual. Contiene HTML con etiquetas de plantilla Django ( `{{ }}` , `{% %}` ) y estilos Bootstrap.
- **Enrutador ( `urls.py` )**: Actúa como despachador. Analiza la URL solicitada en el navegador y deriva la ejecución a la función de vista correspondiente.

### Diagrama de Flujo de una Petición en Proyecto\_BD:

Usuario Navegador → `urls.py` → `views.py` → `models.py` (ORM) → Base de Datos (MySQL/SQLite) → Template (HTML) → Navegador (Pantalla)

## 2. Desglose Archivo por Archivo

### 2.1. `Proyecto_BD/settings.py` (El Cerebro de Configuración)

Este archivo centraliza todas las variables y directivas operativas de Django:

- **INSTALLED\_APPS**: Lista todas las aplicaciones activas. Incluye los módulos nativos ( `django.contrib.auth` para usuarios, `django.contrib.messages` para alertas) y nuestra aplicación personalizada `'productos'`.
- **DATABASES** (Soporte Dual Inteligente):

```

USE_SQLITE = os.environ.get("USE_SQLITE", "").lower() in ("1", "true", "yes")
DB_HOST = os.environ.get("DB_HOST")

if USE_SQLITE or not DB_HOST:
    # Opción A: Local sin Docker -> SQLite3 automático (db.sqlite3)
    DATABASES = {'default': {'ENGINE': 'django.db.backends.sqlite3', 'NAME': BASE_DIR / 'db.sqlite3'}}
else:
    # Opción B: Con Docker -> MySQL 8.0 (contenedor proyecto_bd_mysql)
    DATABASES = {'default': {'ENGINE': 'django.db.backends.mysql', 'NAME': 'bd_productos', ...}}

```

*¿Por qué se hizo así?* Cumple tanto con la recomendación del profesor (usar SQLite3 para facilidad) como con el despliegue profesional contenerizado en Docker con MySQL 8.0.

- **LOGIN\_URL = '/login'** : Indica a Django a qué ruta redirigir automáticamente si un usuario no autenticado intenta ingresar a cualquier vista protegida por **@login\_required**.
- **TIME\_ZONE = 'America/Santiago'** y **LANGUAGE\_CODE = 'es-cl'** : Configura la zona horaria oficial chilena y el idioma español de Chile.

## 2.2. Proyecto\_BD/urls.py (El Enrutador Central)

Contiene la lista `urlpatterns` con todas las rutas URL del sistema:

| Ruta (URL)                                   | Vista Asociada                           | Acción / Requerimiento                                                      |
|----------------------------------------------|------------------------------------------|-----------------------------------------------------------------------------|
| <code>/admin/</code>                         | <code>admin.site.urls</code>             | Panel de administración gráfico oficial de Django.                          |
| <code>/login</code>                          | <code>auth_views.LoginView</code>        | <b>Eta</b> <b>1</b> : Autentica credenciales contra la BD.                  |
| <code>/logout</code>                         | <code>auth_views.LogoutView</code>       | Cierra la sesión del usuario y redirige al login.                           |
| <code>/registro</code>                       | <code>views.registrarUsuario</code>      | <b>Eta</b> <b>3</b> : Formulario de nuevo usuario integrado con hash en BD. |
| <code>/</code>                               | <code>views.mostrarIndex</code>          | Menú principal con accesos y estadísticas.                                  |
| <code>/form_registrar</code>                 | <code>views.mostrarFormRegistrar</code>  | <b>CRUD (Create)</b> : Muestra formulario vacío de producto.                |
| <code>/registrar</code>                      | <code>views.registrarProducto</code>     | <b>CRUD (Create)</b> : Recibe POST e inserta en la BD.                      |
| <code>/listado</code>                        | <code>views.mostrarListado</code>        | <b>CRUD (Read)</b> : Muestra tabla con todos los productos.                 |
| <code>/form_actualizar/&lt;int:id&gt;</code> | <code>views.mostrarFormActualizar</code> | <b>CRUD (Update)</b> : Carga formulario con datos del ID.                   |
| <code>/actualizar/&lt;int:id&gt;</code>      | <code>views.actualizarProducto</code>    | <b>CRUD (Update)</b> : Guarda cambios en el ID indicado.                    |
| <code>/eliminar/&lt;int:id&gt;</code>        | <code>views.eliminarProducto</code>      | <b>CRUD (Delete)</b> : Borra registro por su ID.                            |

*Parámetro dinámico* `<int:id>`: Le indica a Django que capturará un valor numérico de la URL y lo pasará como segundo argumento a la función de la vista.

## 2.3. productos/models.py (El Modelo de Datos)

```
class Producto(models.Model):
    nombre = models.TextField(max_length=100)
    marca = models.TextField(max_length=100)
    precio = models.IntegerField(null=False)

    def __str__(self):
        return f"{self.nombre} ({self.marca}) - ${self.precio}"
```

- **Herencia de `models.Model`**: Le da a la clase todas las capacidades del ORM de Django para mapearse a una tabla en la base de datos (por defecto llamada `productos_producto`).
- **Clave Primaria (`id`)**: No se declara manualmente; Django crea automáticamente una columna `id BIGINT AUTO_INCREMENT PRIMARY KEY`.

- **Campos:** `nombre`, `marca` y `precio` (los 4 atributos exactos exigidos en la Etapa 2 de la pauta).
- **Método** `__str__`: Representación legible en texto para cuando el objeto se visualiza en consola o en el panel de administración.

## 2.4. `productos/views.py` (Lógica de Negocio y Controladores)

### 1. Decorador `@login_required` (Etapa 3 - Seguridad):

Se ubica arriba de cada vista del CRUD. Verifica si `request.user.is_authenticated` es verdadero. Si el usuario no ha iniciado sesión, detiene la ejecución y lo redirige automáticamente a `/login?next=/ruta_solicitada`.

### 2. Registro de Usuarios ( `registrarUsuario` ):

```
def registrarUsuario(request):
    if request.method == 'POST':
        username = request.POST.get('username', '').strip()
        password = request.POST.get('password', '')
        password_confirm = request.POST.get('password_confirm', '')

        # Validaciones: campos obligatorios, coincidencia y usuario existente
        if password != password_confirm:
            return render(request, 'login.html', {'register_error': 'Las contraseñas no coinciden'})
        if User.objects.filter(username=username).exists():
            return render(request, 'login.html', {'register_error': 'El usuario ya existe.'})

        # User.objects.create_user encripta la clave con algoritmo PBKDF2 (SHA-256)
        user = User.objects.create_user(username=username, password=password)
        login(request, user) # Inicia sesión en la cookie
        return redirect('/')
```

### 3. Operaciones CRUD Explicadas:

- **READ (Listar):** `productos = Producto.objects.all().order_by('id')`  
Genera la consulta `SELECT * FROM productos_producto ORDER BY id ASC` y pasa la lista al template en el diccionario de contexto `{'productos': productos}`.
- **CREATE (Registrar):** `Producto.objects.create(nombre=nombre, marca=marca, precio=int(precio))`  
Equivale a `INSERT INTO productos_producto (nombre, marca, precio) VALUES (...)`. Luego usa `redirect('/listado')` para evitar duplicar el envío del formulario.
- **UPDATE (Actualizar):** `producto = get_object_or_404(Producto, id=id)` busca el registro. Si existe, actualiza sus propiedades ( `producto.nombre = ...` ) y ejecuta `producto.save()` (ejecutando un `UPDATE` en SQL).
- **DELETE (Eliminar):** `producto = get_object_or_404(Producto, id=id)` seguido de `producto.delete()` (ejecutando un `DELETE FROM productos_producto WHERE id = ...`).

## 2.5. `productos/templates/` (Vistas y Seguridad en Formularios)

- **Protección CSRF ( `{% csrf_token %}` ):** Es una etiqueta obligatoria de Django dentro de cada formulario `POST`. Genera un token criptográfico único oculto. Al enviar el formulario, el middleware verifica que provenga legítimamente de nuestro sitio y no de un atacante externo (previene ataques de *Cross-Site Request Forgery*).
- **Diseño Centrado (Bootstrap 5):** Se aplicó la clase `d-flex align-items-center justify-content-center min-vh-100` para que todas las pantallas (login, formularios, tablas) queden perfectamente centradas tanto

vertical como horizontalmente en cualquier monitor.

- **Confirmación con JavaScript:** En `listado.html`, el botón de eliminar invoca la función `botonEliminar(id, nombre)` que lanza un cuadro de diálogo nativo `window.confirm(...)` para prevenir eliminaciones accidentales.

### 3. Preguntas Clave para la Interrogación (Simulacro Oral)

---

El docente Javier Gutiérrez evaluará tu comprensión del código (30 puntos de la pauta). Aquí tienes las preguntas más probables y cómo responderlas de forma concisa y técnica:

#### 1. ¿Qué es el ORM de Django y qué ventaja ofrece en este proyecto?

**Respuesta:** Es el *Object-Relational Mapping*. Permite manipular la base de datos utilizando objetos y clases Python (como `Producto.objects.all()` o `.create()`) sin tener que escribir consultas SQL manuales. Esto previene inyecciones SQL y hace que el código funcione igual en SQLite o MySQL sin cambiar una sola línea de lógica.

---

#### 2. ¿Cómo se protegió el acceso a las pantallas del CRUD para que nadie entre sin iniciar sesión?

**Respuesta:** Utilicé el decorador `@login_required` importado de `django.contrib.auth.decorators` sobre cada función de vista en `views.py`. Si alguien intenta ingresar directamente por URL sin estar autenticado, Django intercepta la petición y lo redirige a `/login` gracias a la variable `LOGIN_URL` configurada en `settings.py`.

---

#### 3. ¿Para qué sirve la etiqueta `{% csrf_token %}` en los formularios?

**Respuesta:** Genera un token de seguridad único para prevenir ataques CSRF (Cross-Site Request Forgery). Django exige este token en todas las solicitudes POST; si una petición llega sin este token válido, Django la bloquea con un error 403 Forbidden.

---

#### 4. ¿Cuál es la diferencia entre `render()` y `redirect()` en una vista?

**Respuesta:** `render()` procesa una plantilla HTML inyectándole variables de contexto y devuelve el contenido en la misma petición (usado para mostrar formularios o listas). `redirect()` envía una respuesta HTTP 302 al navegador para que haga una nueva solicitud GET a otra URL (usado después de guardar, editar o borrar para evitar reenvíos de formulario).

---

#### 5. ¿Qué hace la función `get_object_or_404(Producto, id=id)` ?

**Respuesta:** Intenta obtener el registro del modelo `Producto` que coincida con el ID especificado. Si el ID no existe en la base de datos, en lugar de lanzar una excepción que tire el servidor, Django devuelve automáticamente una página de error HTTP 404 Not Found controlada.

---

#### 6. ¿Cómo gestiona el proyecto las contraseñas de los usuarios para que no queden en texto plano?

**Respuesta:** En la vista `registrarUsuario` se utiliza el método nativo `User.objects.create_user(username, password)`. Este método aplica automáticamente un algoritmo de derivación de claves basado en hash criptográfico (PBKDF2 con SHA-256 y sal), por lo que la contraseña nunca se guarda en texto claro en la tabla `auth_user`.

---

#### 7. ¿Qué representan las migraciones en Django y para qué sirven?

**Respuesta:** Son archivos Python (ubicados en la carpeta `migrations/`) que registran el historial de cambios en los modelos. El comando `makemigrations` detecta cambios en `models.py` y crea la receta; el comando `migrate` lee esa receta y ejecuta los comandos DDL (`CREATE TABLE`, `ALTER TABLE`) correspondientes en el motor de BD activo.

---

## 8. ¿Cómo lograste que el proyecto funcione tanto en Docker como en un PC sin Docker?

**Respuesta:** En `settings.py` se evalúa la variable de entorno `DB_HOST`. Si está presente (como en Docker Compose donde `DB_HOST=db`), Django se conecta al contenedor de MySQL 8.0 mediante PyMySQL. Si no está presente, se activa el motor nativo `sqlite3`, permitiendo clonar y ejecutar el proyecto con un simple `python manage.py runserver` sin instalar motores pesados.

---

## 9. ¿Cómo se comunican los contenedores en Docker Compose?

**Respuesta:** Docker Compose crea una red interna donde los contenedores se resuelven entre sí por el nombre del servicio. El servicio web usa el host `db` para llegar a MySQL en el puerto 3306. Además, se configuró un `healthcheck` en MySQL para que el contenedor web espere a que la base de datos esté lista antes de correr las migraciones.

---

## 10. ¿Por qué usamos `enctype` estándar o método POST en el registro de productos?

**Respuesta:** Usamos método `POST` porque estamos alterando el estado de la base de datos (creando o modificando registros). Enviar datos sensibles o de escritura por `GET` expondría los parámetros en la URL y en el historial del navegador.

---